SETS! AND DICTS!

ADMIN STUFF

- Exam scores will come out this afternoon, median score of 70%
  - From my POV, solid performance overall on challenging exam. (The left tail of the distribution is a little longer than expected.)
  - No curve, but Final Clobber Policy means that you can replace this with a higher score if you do better on the final.
  - Regrade requests will open Monday-Friday of next week.
- Sunday Review Sessions every Sunday @ 3pm if you want consistent exam practice
- HW4 due Wednesday, March 4

REVIEW: SETS

Sets look similar to lists, but with `{}` instead of `[]`

- `{"this"}`
- `{"howdy", "partner"}`

Cannot store lists, dicts or other sets within a set

Most important part of a set: it enforces uniqueness of its elements. An element can only be in the set once or not at all.

```
primary_colors = {"red", "green", "blue"}   # a set of strs
some_primes = {3, 103, 5, 23, 2}            # a set of ints
```

REVIEW: SETS

Sets are an **unordered** container for data.
Unordered means:

- We can still use `for` to loop over every element
- can still use `in` to see if something is in it
- can **not** access into it with an index and `[]` or slice

DEMO

```
some_primes = {3, 103, 5, 23, 2}                # a set of ints
for prime in some_primes:
    print(prime)
```

...could print `3 5 23 2 103` or `3 103 5 23 2` or `5 2 1 103 23`!

DEMO

```
some_primes = {3, 103, 5, 23, 2}                # a set of ints
print(4 in some_primes)                         # prints False
print(5 in some_primes)                         # prints True
```

DEMO

```
some_primes = {3, 103, 5, 23, 2}                # a set of ints
first_two_primes = some_primes[:2]
```

...leads to a `TypeError: 'set' object is not subscriptable.`

MORE SET REVIEW

There are a few common features of a set:

- `.add()` adds an item to the set
- `.remove()` removes an item from a set, but crash if the item is not in the set
- `.discard()` removes an item from a set, ignore if the item is not in the set
- set comprehension work like list comprehensions, use {} instead of []

GAME TIME!

I need two teams of two.

For each round, one player from each team will stand at the table. As soon as you know the answer, knock on the table. I will call on you, and you can answer for a point. If you get it wrong, the other team has a chance to steal.

ROUND 1

ROUND 1

What gets printed? Why?

```
s = {"yes", "no", "I", "don't", "think", "so"}
print(len(s))
```

ROUND 2

ROUND 2

What gets printed? Why?

```
s = {"yes", "yes,", "no", "I", "don't", "no"}
print(len(s))
```

ROUND 3

ROUND 3

What gets printed? Why?

```
s = {"apple", "carrot", "pear", "grape", "banana"}
t = set()
for fruit in s:
    t.add(len(fruit))
print(len(t))
```

ROUND 4

Reminder: `set(seq)` creates a new set containing all of the unique elements from the sequence `seq`.

ROUND 4

What gets printed? Why?

```
s = set("mississippi")
print(len(s))
```

ROUND 5

`suspects` is a set that models the name of current suspects for a crime. It's updated over time as new evidence comes to light.

After all evidence has come to light, who are the remaining suspects? (Or, does the program crash?)

ROUND 5

After all evidence has come to light, who are the remaining suspects? (Or, does the program crash?)

```
suspects = set()
suspects.add("Dunne")
suspects.add("Calvino")
suspects.add("Cole")
suspects.add("Wallace")
suspects.remove("Calvino")
suspects.remove("Wallace")
suspects.add("Dunne")
print(suspects)
```

CHALLENGE ROUND

Working together, help solve the following problem.

> *You have a list of the names of everyone who signed in to come see a free art exhibit throughout a week. After the gallery closed for the weekend, you noticed that someone left a jacket behind. You can see that their name is "Anna". You want to figure out which people you should try reaching out to to return the wallet.*

CHALLENGE ROUND

Write the body of this function! (Play along at home in **C12** so you can help score.)

```
def find_possible_matches(visitors: list[str], name: str) -> set[str]:
    """Given a list of names of all visitors to a gallery and a
    name of a person of interest, return a set of names that might
    correspond to the person who left their jacket!
    """
    # TODO!


# usage:
vs = ["Anna Marina", "Edoardo Cerentano", "Franceso Di Salvi",
      "Anna Cerentano", "Anna Marina", "Teodoro Marina"]
name = "Anna"
find_possible_matches(vs, name) # {"Anna Marina", "Anna Cerentano"}
```

SET OPERATIONS

NAME	MEANING	METHOD	OPERATOR
Union	Create a new set with all elements from both	`s.union(t)`	`s | t`
Intersection	Create a new set with only elements that appear in both sets	`s.intersection(t)`	`s & t`
Difference	Create a new set with only elements in `s` that don't appear in `t`	`s.difference(t)`	`s - t`
Symmetric Difference	Create a new set with elements that appear in only one set *but not both*	`s.symmetric_difference(t)`	`s ^ t`

SET OPERATIONS PRACTICE

C14

Implement both a union function without using the built-in union operator/function.

```
def set_union(s1, s2):
    """Given two sets, return a new set that has all elements of both
    input sets, e.g.:
    set_union({"hi","ho"}, {"bad","hi"}) -> {"hi", "ho", "bad"}"""
```

DICTIONARIES

Dictionaries (also called "dicts") are the much more commonly used **unordered** collection

- Associates **keys** to **values**
- Allow for looking up some information associated with a search key
- Keys must be unique, values do not need to be unique

WHAT IS A MAPPING?

Any association from **keys** (things you can search by) to **values** (information you might want to know.)

The Penn Directory, for example:

```
Name : Email
------------
Harry Smith : sharry@seas
Travis McGaha : tqmcgaha@seas
...
```

Here, the names are keys and the emails are values.

DICT SYNTAX

Dict literals are defined with curly braces (`{}`) and separate keys and values with a colon.

- `{3, 10, 15}`
  - is a **set** with three elements
- `{"Harry" : "sharry", "Talie" : "taliem"}`
  - is a **dict** with two elements (key-value pairs)
- `{}` is an empty dict
  - writing just `dict()` gets the same result

DICTIONARY PRACTICE: READING

Given the following dictionaries, which ones are legal dictionaries? (Legal / Illegal)

S7

```
speak = {
  "dog": "woof",
  "cat": "meow",
  "seal": "arf",
  "fox": "woof"
}
```

S8

```
movies = {"F1", "Wicked 2",
          "Frankenstein"}
```

S9

```
recs = {207:["Zwe", "Cam"],
        203: ["Elliot", "Derek"],
        201: ["Cam", "Zwe"]}
```

GETTING

Given a dictionary `d`, you can access the value associated with key `k` by writing `d[k]`:

```
last_names["Harry"]                          # ---> "Smith"

area_codes[215]                              # ---> "Philadelphia"

months["October"]                            # ---> 10

food_group["Milk"]                           # ---> "Dairy"
servings_per_day["Dairy"]                    # ---> 3
servings_per_day[food_group["Milk"]]         # ---> 3 (do you see why?)
```

Any of these would throw an error if the key is not already present in the dictionary!

SETTING

Given a dictionary `d`, you can set or update the value associated with key `k` by writing `d[k] = <new_value>`:

```
last_names["Talie"] = "Massachi"

recitation_attendances["Renate"] = 4

total_score["Harry"] = round_one_score["Harry"] + \
                       round_two_score["Harry"]

# these are equivalent ways of updating
recitation_attendances["Aksel"] += 1
recitation_attendances["Aksel"] = recitation_attendances["Aksel"] + 1
```

We want to write a function that turns a list of names into a dictionary where the *keys* are the first initials and the *values* are counts of names that start with that letter.

S10 Write out the dictionary that should be returned by this function when called on the list below.

```
count_names_by_letter(["Susan", "Sally", "Paul", "Nico"])
```

We want to write a function that turns a list of names into a dictionary where the *keys* are the first initials and the *values* are counts of names that start with that letter.

What's wrong with this implementation? (**L11**; describe, don't fix)

```
def count_names_by_letter(names: list[str]) -> dict[str, int]:
    initial_counts = dict()
    for name in names:
        first_letter = name[0]
        initial_counts[first_letter] += initial_counts[first_letter]
    return initial_counts
```

(Then, we'll fix it together.)

Previously, `initial_counts[first_letter] += initial_counts[first_letter]` leads to a `KeyError` if `first_letter` isn't already present.

```
def count_names_by_letter(names: list[str]) -> dict[str, int]:
    initial_counts = dict()
    for name in names:
        first_letter = name[0]
        if first_letter not in initial_counts:
          initial_counts[first_letter] = 1
        else:
          initial_counts[first_letter] += initial_counts[first_letter]
    return initial_counts
```

Two cases now: set a *default value* if the key isn't already present; update it if it is.

DESCRIBE THE ISSUE:

We want to write a function that turns a list of names into a dictionary where the *keys* are the first initials and the *values* are lists of names that start with that letter.

```
["Susan", "Sally", "Paul", "Nico"]
🔽
{"S" : ["Susan", "Sally"],
 "P" : ["Paul"],
 "N" : ["Nico"]
}
```

We want to write a function that turns a list of names into a dictionary where the *keys* are the first initials and the *values* are lists of names that start with that letter.

This implementation has a similar problem. Can you fix it? **C12**

```
def group_names_by_letter(names: list[str]) -> dict[str, list[str]]:
    names_by_letter = dict()
    for name in names:
        first_letter = name[0]
        names_by_letter[first_letter].append(name)
    return names_by_letter
```

Make sure to watch the "CSV Parsing Demo" at the end of the "Dicts!" playlist for some help getting started on HW4!